



01_frogger-main(1)...
File



Lab_4_VibeCoding...
PDF

https://github.com/reetporwal66/01_frogger

Read the README and inspect the existing Frogger code. Implement only Task 1: Fix Vehicle Collision Detection. The current collision logic compares the vehicle's grid column with the frog's grid column, but vehicles move continuously in pixel space and can have different widths. Modify the collision detection so that a collision is detected reliably when the frog's rectangle overlaps with a vehicle's rectangle. Use the existing `get_rect()` methods and preserve the current project structure. Do not implement or modify the lives/respawn system, goal/score system, timer, or any other tasks. After making the change, provide the complete modified code and briefly explain what was changed and why.

Worked for 23s

Analyzed

Analyzed

Analyzed

Done. I inspected the README and the existing Frogger code. The README confirms **Task 1 is only vehicle collision detection**, while Tasks 2–4 are separate features.  Lab_4_VibeCoding_Student_handout

What I changed

Only this file was modified:

`frogger/game/collisions.py`

The old logic did this:

`</>` Python

```
vehicle_col = int(v.x // CELL_SIZE)
if vehicle_col == frog.col and v.row == frog.row:
    return True
```

That was unreliable because vehicles move continuously in **pixel coordinates** and have different widths (40 or 70 pixels).

I changed it to use the existing `get_rect()` methods:

`</>` Python

```
frog_rect = frog.get_rect(CELL_SIZE)
vehicle_rect = v.get_rect(CELL_SIZE)

if frog_rect.colliderect(vehicle_rect):
    return True
```

So collision now happens when the **actual frog rectangle overlaps the actual vehicle rectangle**, regardless of the vehicle's exact pixel position or width.

Complete modified `collisions.py`

</> Python

```
"""
collisions: frog-vs-vehicle collision detection.
"""

CELL_SIZE = 50

def check_collision(frog, vehicles):
    """
    Returns True if the frog rectangle overlaps any vehicle rectangle.
    """
    frog_rect = frog.get_rect(CELL_SIZE)

    for v in vehicles:
        vehicle_rect = v.get_rect(CELL_SIZE)
        if frog_rect.colliderect(vehicle_rect):
            return True

    return False
```

Verification

-  README inspected
-  Existing project structure preserved
-  Only `collisions.py` changed
-  Existing `Frog.get_rect()` used
-  Existing `Vehicle.get_rect()` used
-  Pixel-level rectangle overlap detection implemented
-  Python files compile successfully
-  No lives/respawn changes
-  No goal/score changes
-  No timer changes
-  No other tasks modified

I also packaged the **Task 1-only modified project**:

 [Download the modified Frogger project](#)

For your assignment, this should be the **separate Task 1 commit** before you work on Task 2. The handout specifically requires separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#) 

Implement only Task 2 from the Frogger README: add a 3-life system with appropriate frog respawn behavior after a vehicle collision. Integrate it with the existing collision detection from Task 1. The player should start with 3 lives; each collision should decrease the remaining lives by 1 and reset the frog to its starting position. Display the current number of lives during gameplay. When all 3 lives are lost, show an appropriate Game Over state and allow the player to restart the game using the existing R restart control. Preserve the existing movement and collision functionality. Do not implement Task 3 (goal/score/win state) or Task 4 (30-second timer). Modify only the files necessary for this task and explain each change.

Worked for 34s

Done — I implemented **only Task 2** on top of your Task 1 version.

Changes made

Only `frogger/game/game_engine.py` was modified.

`collisions.py` from Task 1 is preserved unchanged, so the rectangle-based collision detection is still being used.

1. Added 3 lives

The game now starts with:

 Python

```
self.lives = 3
self.game_over = False
```

2. Collision → lose one life + respawn

The existing Task 1 collision check is used:

 Python

```
if check_collision(self.frog, self.vehicles):  
    self.lives -= 1  
    self.frog.reset()
```

So after every vehicle collision:

- Life decreases by 1
- Frog returns to its starting position

3. Game Over after 3 collisions

When lives reach zero:

</> Python

```
if self.lives == 0:  
    self.game_over = True
```

The game then stops updating/moving vehicles.

4. Lives displayed during gameplay

</> Python

```
renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 10))
```

You'll see Lives: 3 , Lives: 2 , etc.

5. Game Over state

</> Python

```
renderer.draw_banner(surface, font, "GAME OVER - Press R to restart")
```

6. R restarts everything

Pressing R :

- restores 3 lives
- clears Game Over
- rebuilds the frog and vehicles
- puts the frog back at the starting position

Also, while Game Over is displayed, arrow-key movement is disabled; R remains available.

Task 2 complete code — game_engine.py

</> Python

"""

GameEngine: owns the frog and all vehicles, and runs one frame's worth of game logic.

Starter version: the frog can move, hop across the road, and reach the goal - but there's no lives system, no score, and no timer. Collision detection also has a known bug (see game/collisions.py) that Task 1 asks you to fix.

"""

```
import random
```

```
import pygame
```

```
from game.frog import Frog
```

```
from game.vehicle import Vehicle
```

```
from game.collisions import check_collision
```

```
from game.renderer import (
```

```
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH, HEIGHT,
```

```
)
```

What was not implemented

- **✗** Task 3 — goal/score/win system
- **✗** Task 4 — 30-second timer
- **✗** Changes to Task 1 collision detection
- **✗** Changes to frog movement
- **✗** Changes to vehicle movement

The README explicitly separates these into Tasks 2, 3, and 4, and requires separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

Task 2 ZIP

 [Download the Task 2 modified project](#)

One limitation: I successfully ran Python compilation, but this environment doesn't have `pygame` installed, so I couldn't launch the graphical game here for an actual gameplay test. The Task 2 logic itself was checked structurally; you should run the game locally before committing.

For GitHub: make this as a **separate Task 2 commit**, after your Task 1 commit.

Implement only Task 3 from the Frogger README: add a goal/finish condition, score tracking, and a win state. The player should be able to reach the goal area at the top of the board, receive a score when the goal is successfully reached, and see an appropriate Win/Game Won state. Preserve the existing Task 1

vehicle collision detection and Task 2 three-life system, including frog respawn, lives display, Game Over state, and R restart behavior. Do not implement Task 4 (30-second timer). Modify only the files necessary for Task 3 and explain the changes made. Provide the complete contents of every modified file without truncating or omitting any lines.

Worked for 19s

Implemented **only Task 3** on top of your Task 2 version.

The README defines Task 3 as **goal completion + score tracking + an appropriate win state**. It does not specify an exact score value, so I used **100 points for successfully reaching the goal**. Task 4's timer remains untouched.

Files modified

Only:

⟨/⟩ Plain text

```
frogger/game/game_engine.py
```

No changes were made to:

- collisions.py — Task 1 rectangle collision remains intact
- frog.py — Task 2 respawn behavior remains intact
- vehicle.py
- renderer.py
- main.py

What changed

1. Added score

⟨/⟩ Python

```
self.score = 0
```

The score starts at 0.

When the frog reaches GOAL_ROW :

⟨/⟩ Python

```
self.score += GOAL_SCORE  
self.game_won = True
```

I used:

</> Python

```
GOAL_SCORE = 100
```

2. Added Win state

</> Python

```
self.game_won = False
```

Once the frog reaches the top goal:

</> Python

```
self.game_won = True
```

The game then stops updating, just like Game Over.

3. Score is displayed

The existing renderer's `draw_text()` is reused:

</> Python

```
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 34))
```

So gameplay shows both:

</> Plain text

```
Lives: 3  
Score: 0
```

and after reaching the goal:

</> Plain text

Score: 100

GAME WON! - Press R to restart

4. R restart works for both Game Over and Win

I changed the restart handling so R is checked first:

Python

```
if key == pygame.K_r:
    self.lives = 3
    self.score = 0
    self.game_over = False
    self.game_won = False
    self._build_entities()
    return
```

Therefore:

- Game Over → R → new game
- Game Won → R → new game
- Lives → 3
- Score → 0
- Frog → starting position
- Vehicles → rebuilt

5. Task 2 is preserved

Vehicle collision still uses the Task 1 implementation:

Python

```
if check_collision(self.frog, self.vehicles):
    self.lives -= 1
    self.frog.reset()
```

So collisions still:

1. Detect using rectangle overlap
2. Remove one life
3. Respawn the frog
4. Show Game Over at 0 lives

6. No Task 4

There is **no timer/countdown/timeout logic**.

Complete modified frogger/game/game_engine.py

</> Python

"""

GameEngine: owns the frog and all vehicles, and runs one frame's worth of game logic.

Task 2 adds a three-life system with respawn after vehicle collisions.

Task 3 adds score tracking and a win state when the frog reaches the goal.

"""

import random

import pygame

from game.frog import Frog

from game.vehicle import Vehicle

from game.collisions import check_collision

from game.renderer import (
 GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH, HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2] # one entry per road row, alternating direction

Task 3 behavior now

</> Plain text

Start

├─ Lives = 3

└─ Score = 0

|

▼

Move frog normally

|

├─ Vehicle collision

|

├─ Lives - 1

|

└─ Frog respawns

|

└─ Reach top goal

|

├─ Score +100

└─ GAME WON

|

└─ R → restart

I also verified that the modified Python files compile successfully.

📄 Download the complete Task 3 project

For your lab, make this a **separate Task 3 commit** after your Task 2 commit. The handout specifically asks for separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

The code is still truncated. Please provide the **complete final contents** of `frogger/game/game_engine.py` from the first line to the last line. Do not omit, summarize, compress, or replace any part with `...`. I need the entire file so I can replace my existing file exactly. Keep Task 1 and Task 2 unchanged and implement only Task 3

Worked for 9s

You're right. I checked the **actual Task 2 `game_engine.py`** from the project rather than using the previously generated version. Below is the **entire file from the first line to the last line, with only Task 3 additions**.

Task 1 collision detection and Task 2 lives/respawn behavior are preserved exactly. The only Task 3 changes are:

- `score = 0`
- `game_won = False`
- reaching `GOAL_ROW` awards 100 points and sets `game_won`
- score display
- Win message
- `R` resets score and win state
- movement/update stops after winning

Python

```
"""
GameEngine: owns the frog and all vehicles, and runs one frame's worth
of game logic.
```

```
Starter version: the frog can move, hop across the road, and reach the
goal - but there's no lives system, no score, and no timer. Collision
detection also has a known bug (see game/collisions.py) that Task 1
asks you to fix.
```

```
"""
```

```
import random
```

```
import pygame
```

```
from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH, HEIGHT,
```

Exactly what changed from your Task 2 file

Part	Task 2	Task 3
Lives	3	Unchanged
Collision	Task 1 rectangle collision	Unchanged
Respawn	self.frog.reset()	Unchanged
Game Over	0 lives	Unchanged
Score	—	Added, starts at 0
Goal	Frog was reset at GOAL_ROW	+100 score and Win state
Win state	—	Added
R after Game Over	Restart	Unchanged
R after Win	—	Restarts
Timer	None	Still none — Task 4 not implemented

One important correction from my previous response: **the original Task 2 file did not have the expanded comments/formatting I previously showed**. The code above is based directly on the actual Task 2 project file and keeps its existing structure, with only the Task 3 additions.

Implement only Task 4 from the Frogger README: add a 30-second countdown timer to the game. The timer should start at 30 seconds when a new game begins, count down during gameplay, and be displayed clearly during gameplay. If the timer reaches 0 before the frog reaches the goal, the game should enter an appropriate Game Over/Time Up state. Preserve all existing functionality from Tasks 1–3: rectangle-based vehicle collision detection, 3 lives with respawn, score tracking, goal completion,

win state, and R restart behavior. When R is pressed after Game Over, Time Up, or Win, restart with 3 lives, score 0, and a fresh 30-second timer. Do not change the existing Task 1–3 behavior except where necessary to integrate the timer. Modify only the files necessary for Task 4 and provide the complete contents of every modified file without truncating or omitting any lines.

For **Task 4**, only `frogger/game/game_engine.py` needs to change. The timer uses `pygame.time.get_ticks()` so it counts actual elapsed gameplay time rather than depending on frame rate.

Below is the **complete file from first line to last line**, based on the Task 3 version above. Task 1–3 behavior is otherwise preserved.

</> Python

```
"""
```

```
GameEngine: owns the frog and all vehicles, and runs one frame's worth
of game logic.
```

```
Starter version: the frog can move, hop across the road, and reach the
goal - but there's no lives system, no score, and no timer. Collision
detection also has a known bug (see game/collisions.py) that Task 1
asks you to fix.
```

```
"""
```

```
import random
```

```
import pygame
```

```
from game.frog import Frog
```

```
from game.vehicle import Vehicle
```

```
from game.collisions import check_collision
```

```
from game.renderer import (
```

```
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH, HEIGHT,
```

```
)
```

Task 4 additions

The new state is:

</> Python

```
TIME_LIMIT = 30
```

```
self.time_up = False
```

```
self.time_remaining = TIME_LIMIT  
self.start_time = pygame.time.get_ticks()
```

During each active frame, elapsed time is calculated and subtracted from 30:

</> Python

```
elapsed_seconds = (pygame.time.get_ticks() - self.start_time) // 1000  
self.time_remaining = max(0, TIME_LIMIT - elapsed_seconds)
```

At zero:

</> Python

```
if self.time_remaining == 0:  
    self.time_up = True  
    return
```

and the player sees:

</> Plain text

TIME UP! - Press R to restart

R now resets all four tasks' relevant state:

</> Plain text

```
Lives → 3  
Score → 0  
Timer → 30  
Game Over → False  
Time Up → False  
Win → False  
Frog/vehicles → rebuilt
```

Task 1's collision code was **not changed**, Task 2's life/respawn logic was **not changed**, and Task 3's scoring/goal/win logic was **not changed** except that active gameplay now also stops when time expires.